

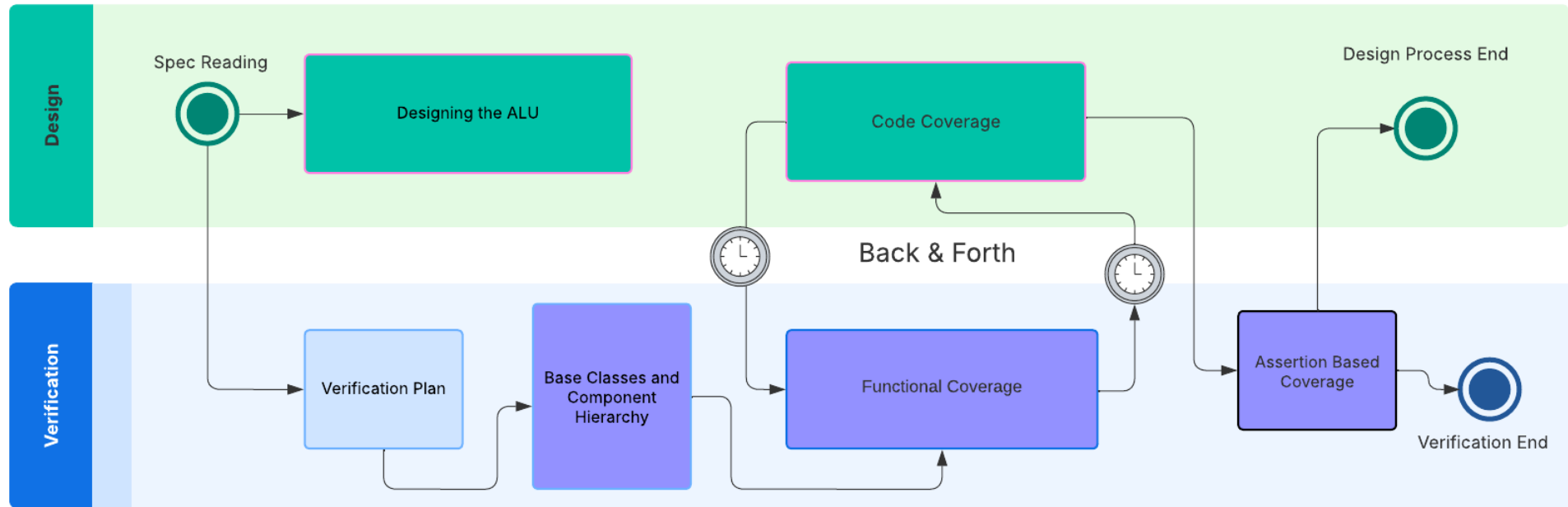


Design & Verification of an ALU using SystemVerilog & SVA

*Abdelrahman Mohamed
Yassien*

dir: /home/svgpdv25abohamad/alu_project

Project Timeline



Verification Plan

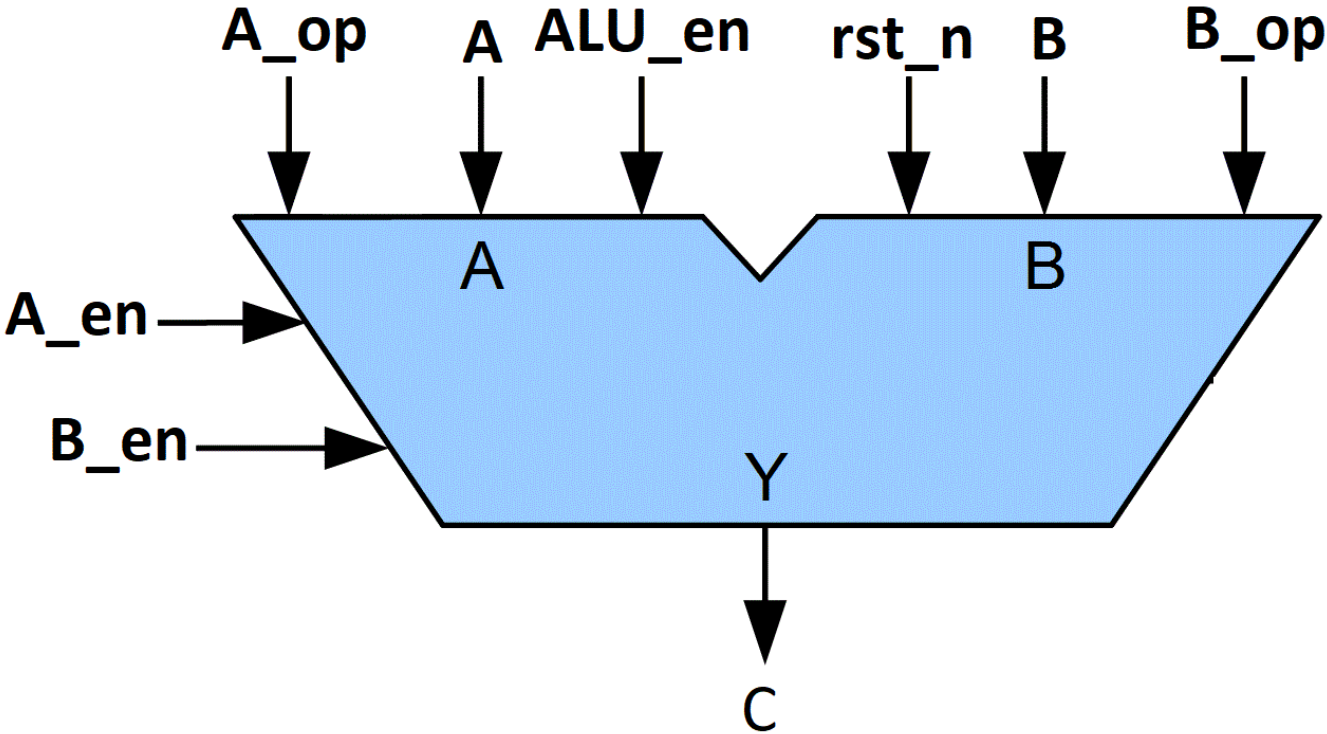
Plan	Description
Reset DUT	Resetting the DUT
ALU_en	Sending transactions while ALU_en is off
A_op Opcodes	Checking all A_op opcodes when (A_en & ~B_en)
B_11op Opcodes	Checking all B_op opcodes when (A_en & B_en)
Random Reset	Randomly Resetting the DUT during the entire simulation
Code Coverage	Ensuring all the DUT's code lines were HIT using code coverage and making better test cases if not.
Functional Coverage	Ensuring all opcodes/values are met and tried using Functional Coverage and Fixing the DUT if not
Error Injection	Using an Error Injection scenario to check if the DUT hangs or behaves in a destructive way if faced with an undefined state or an out-of-bounds/unexpected inputs
Assertion Based Coverage	Asserting over the design specs, clock period, alu operations, alu_en, rst_n

ALU DESIGN SPECIFICATIONS

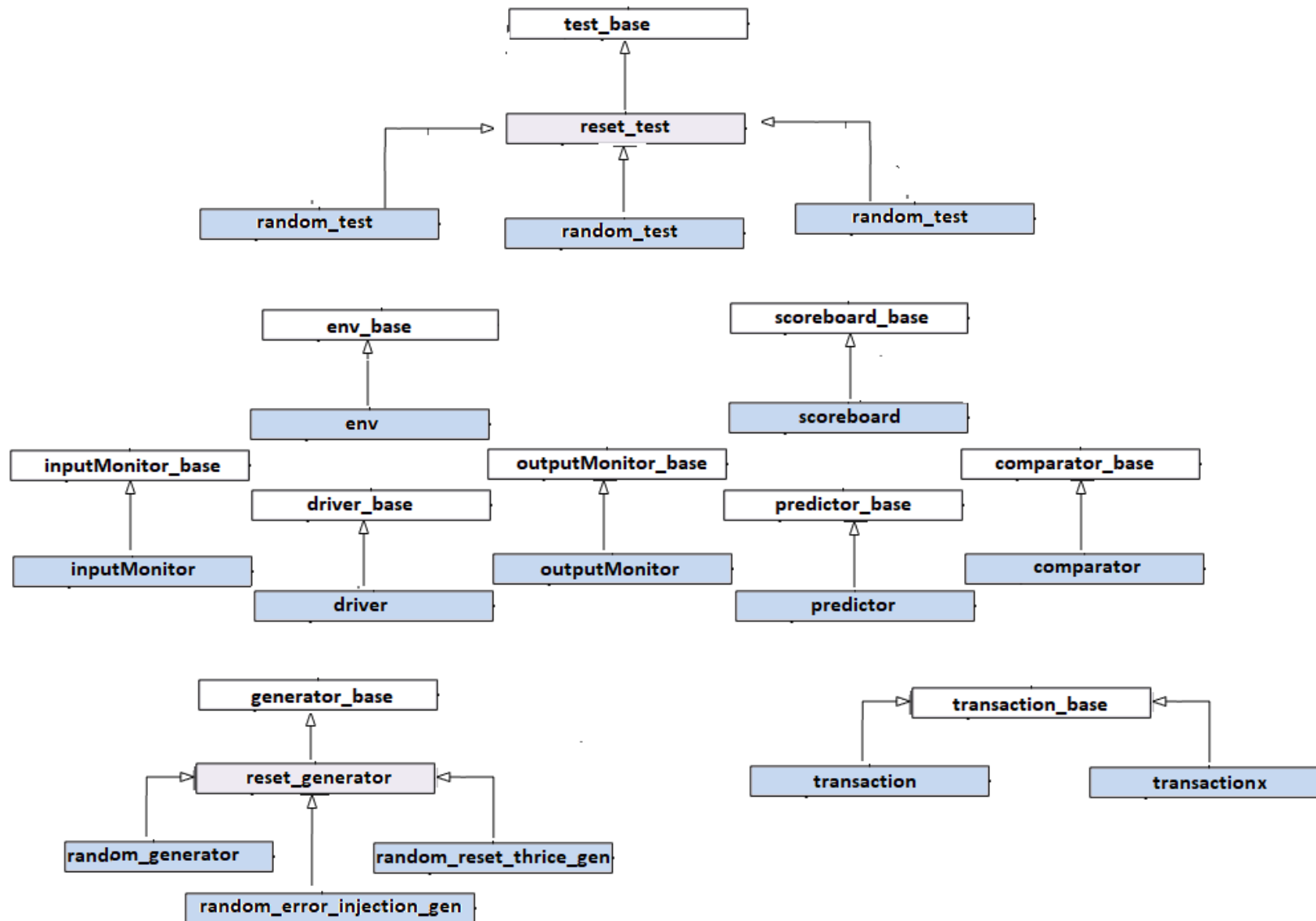
(A_EN & ~B_EN)	A_op	C = ?	Description
	0	A + B	Sum of inputs
	1	A – B	B subtracted from A
	2	A ^ B	A bitwise XOR B
	3	A & B	A bitwise & B
	4	A & B	A bitwise & B
	5	A B	A bitwise OR B
	6	~(A ^ B)	A XNOR B
	7	NULL	Not permitted to take this value

(A_EN & B_EN)	B_op	C = ?	Description
	0	A + B	Sum of inputs
	1	A – B	B subtracted from A
	2	A ^ B	A bitwise XOR B
	3	A & B	A bitwise & B

(~A_EN & B_EN)	B_op	C = ?	Description
	0	~(A & B)	A NAND B
	1	A + B	Sum of inputs
	2	A + B	Sum of inputs
	3	NULL	Not permitted to take this value



Verification Environment Classes



TOP ALU TESTBENCH

RANDOM TEST

R

RANDOM RESET THRICE TEST

GENERATOR

DRIVER

INPUT
MONITOR

SCOREBOARD

Coverage
Collector

OUTPUT
MONITOR

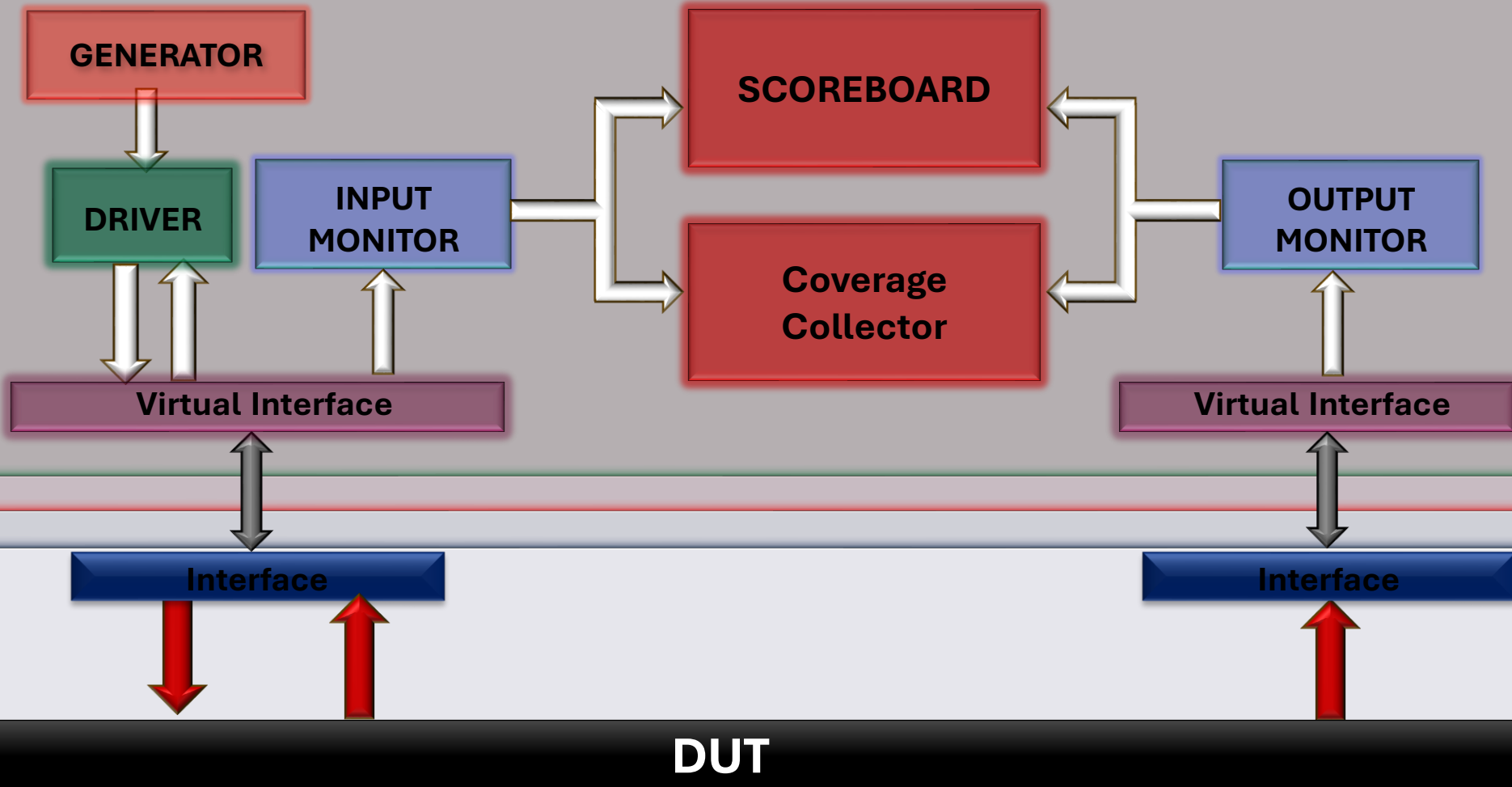
Virtual Interface

Virtual Interface

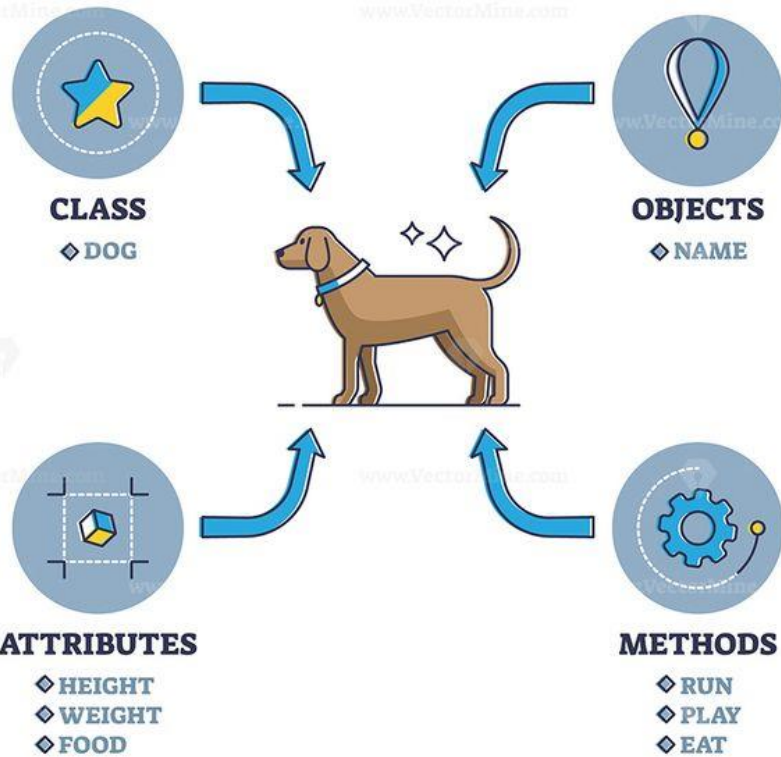
Interface

Interface

DUT



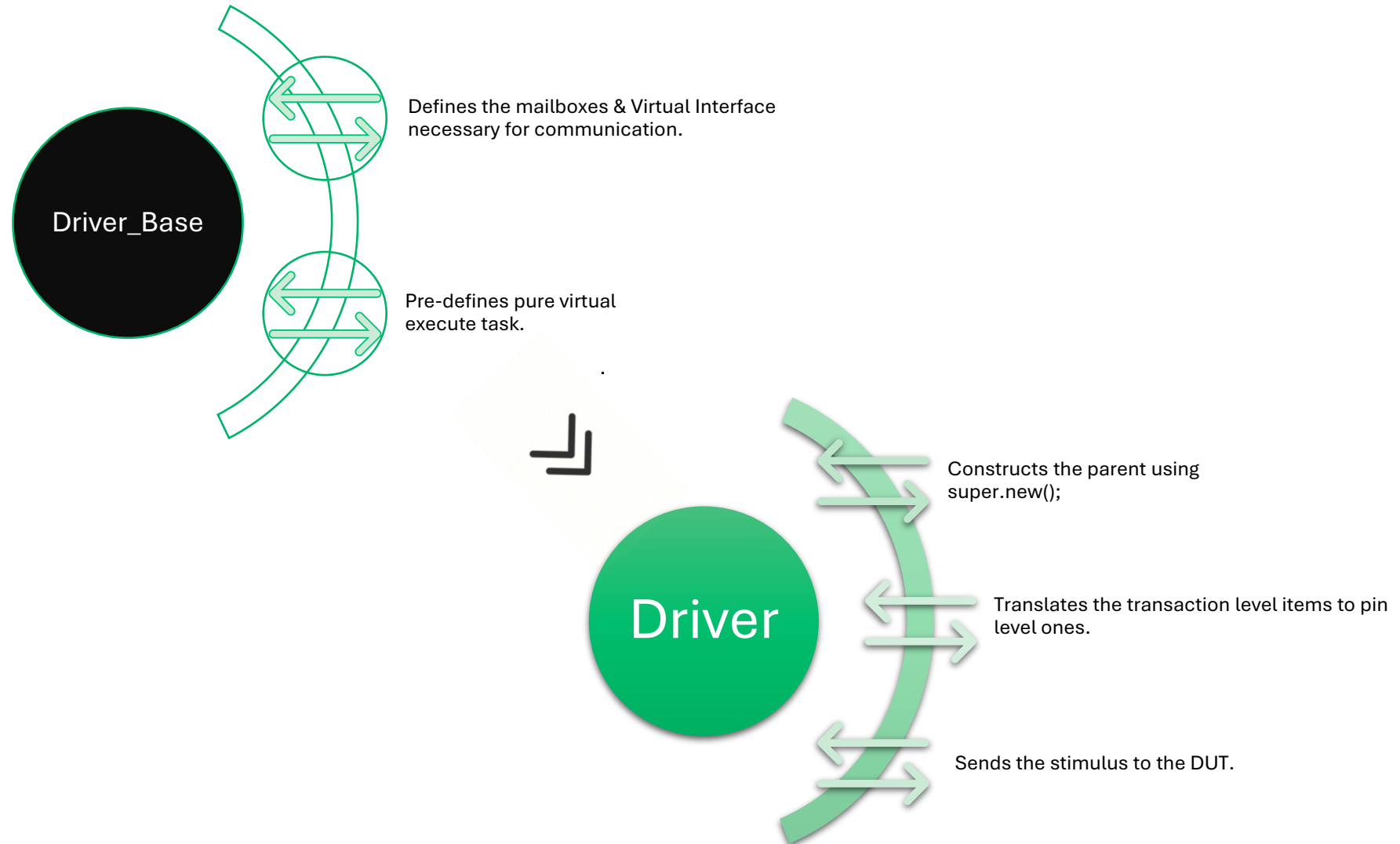
OBJECT ORIENTED PROGRAMMING



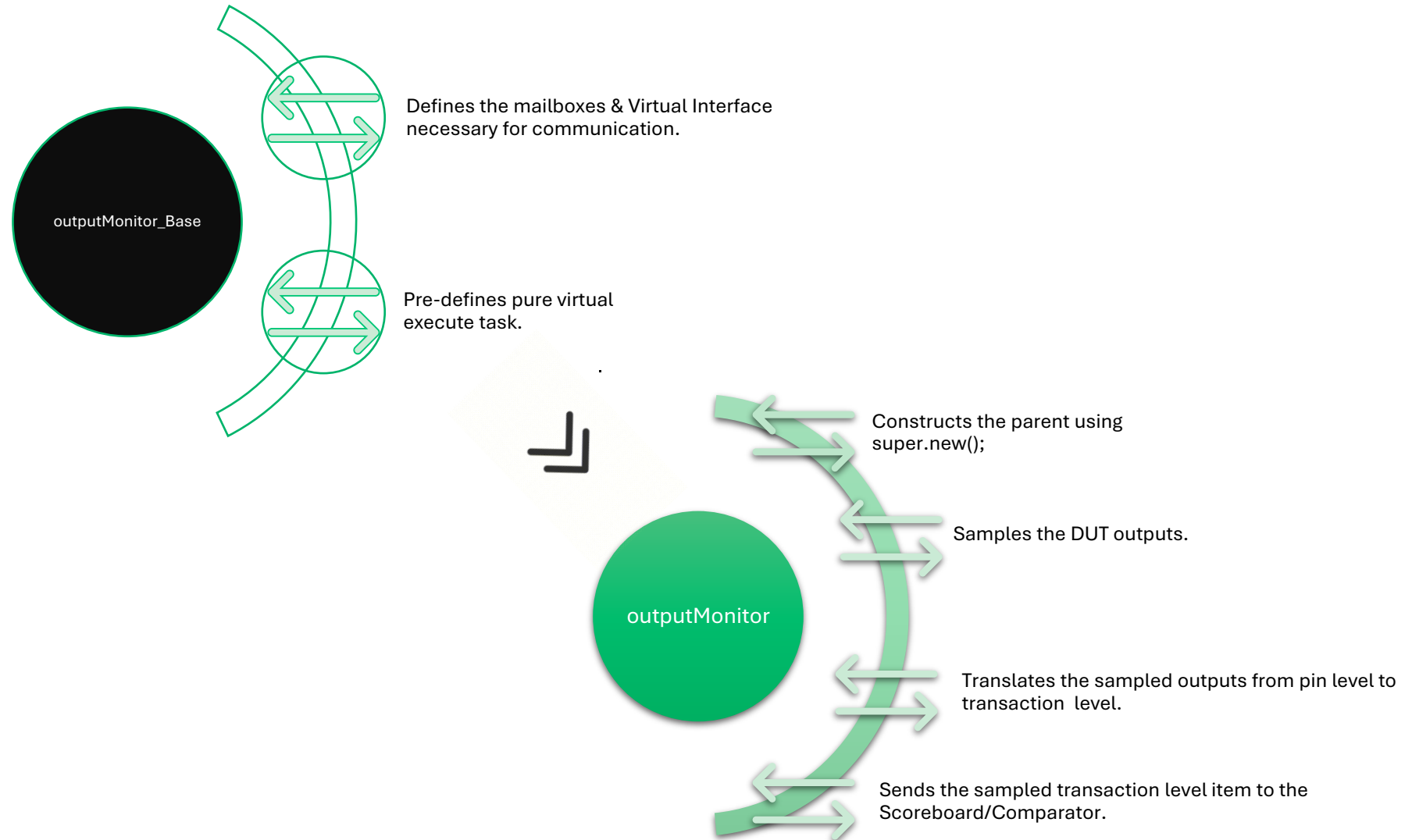
Object oriented programming

Classes? What do they do?

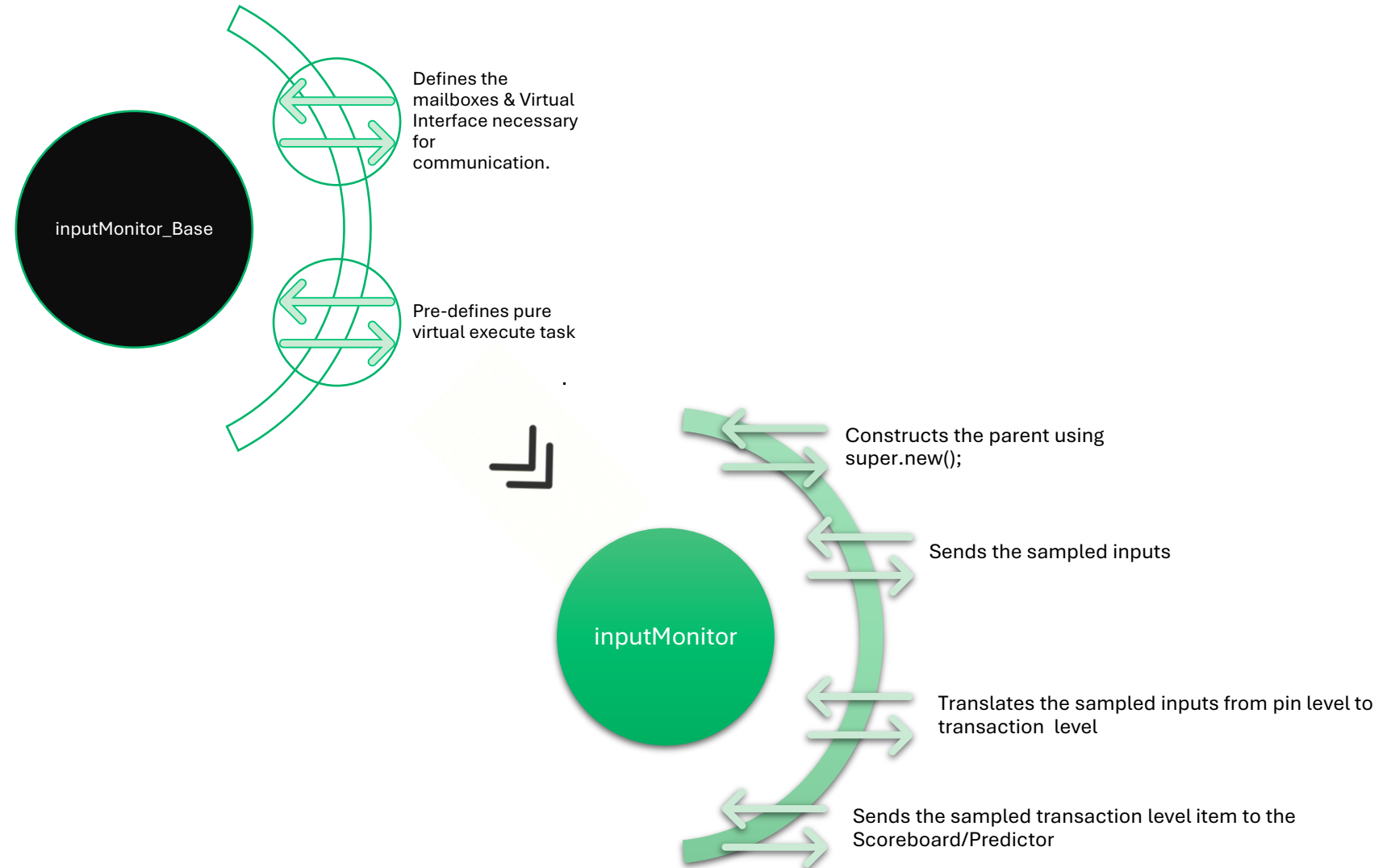
Driver & Driver Base



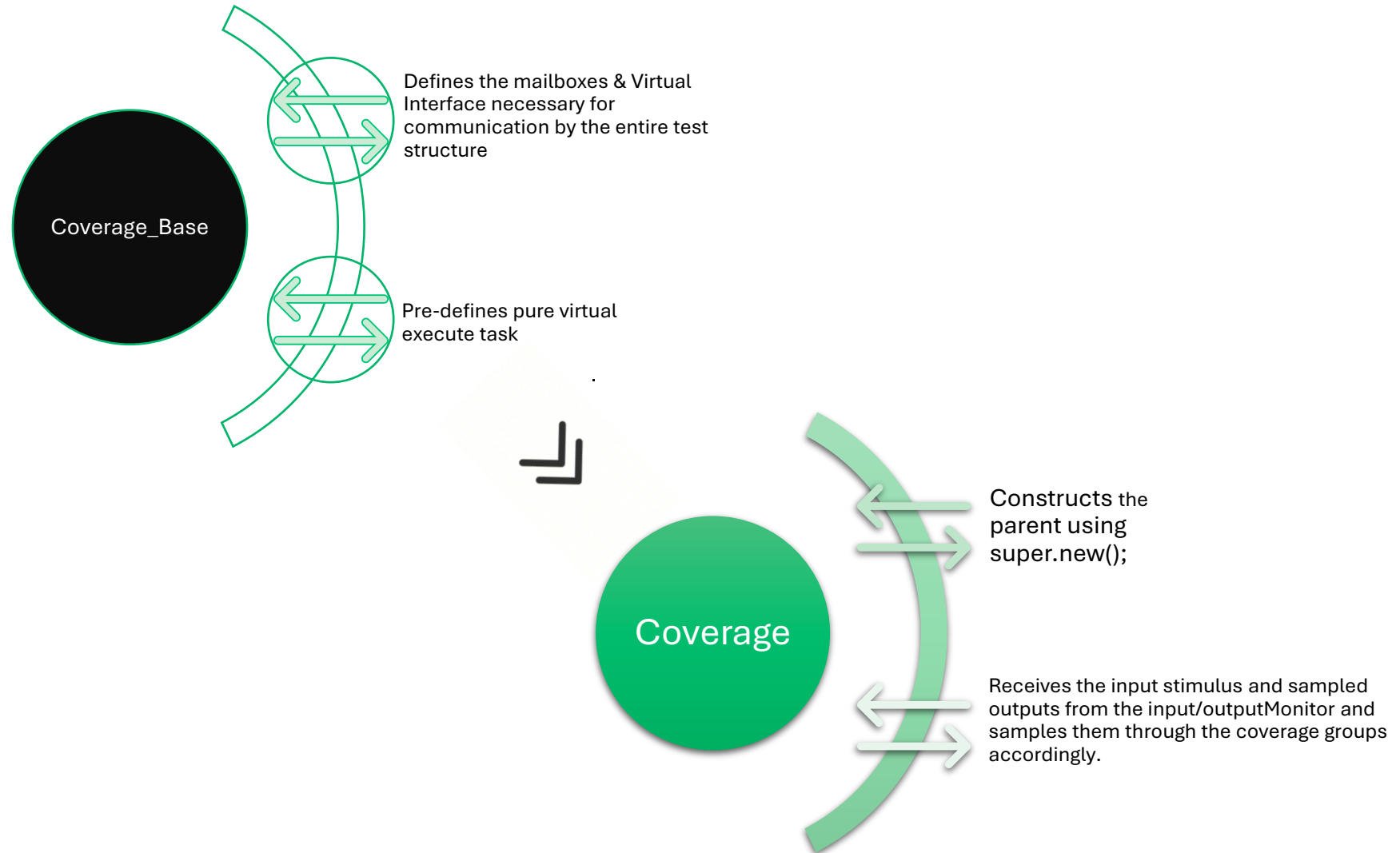
outputMonitor & outputMonitor Base



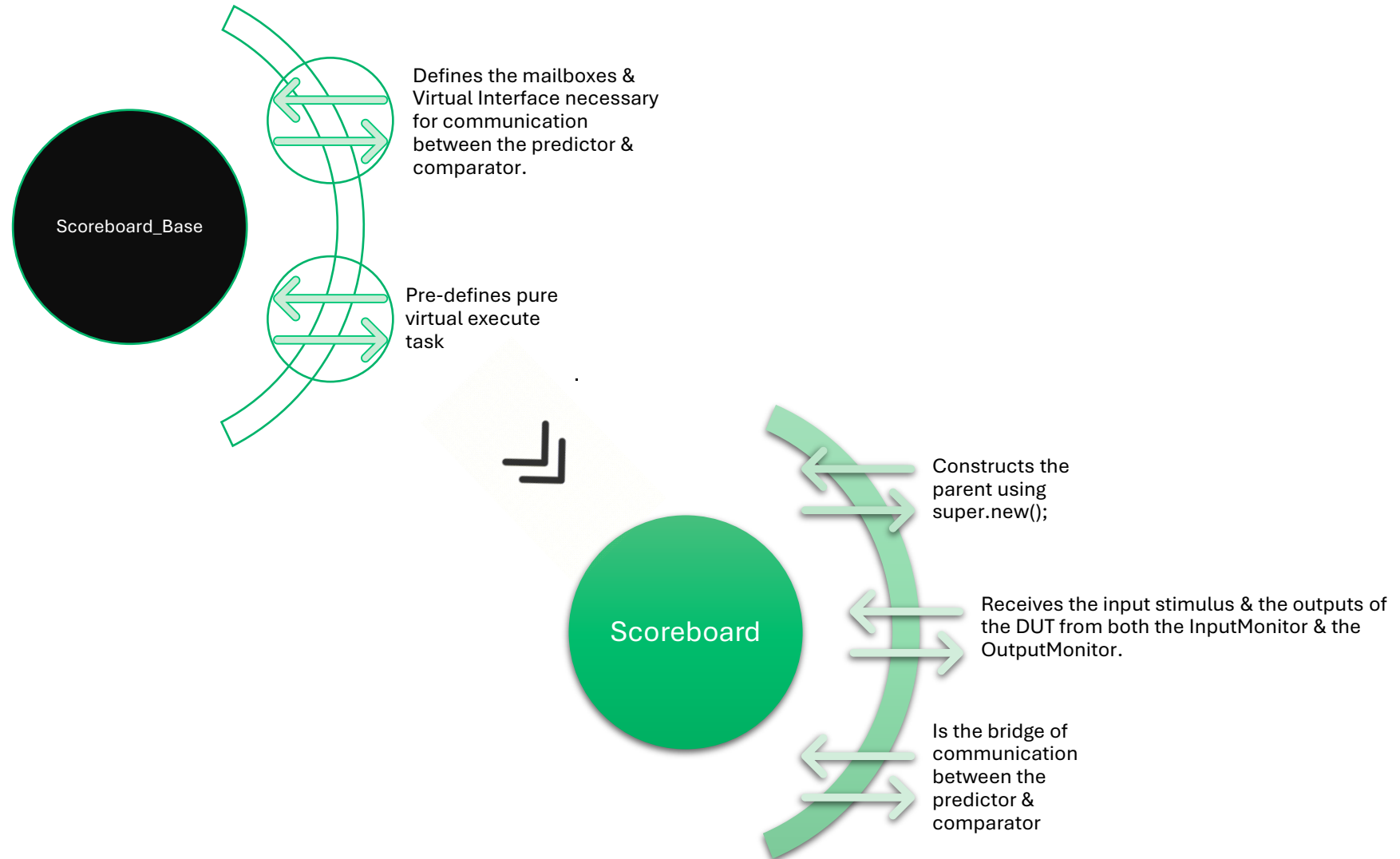
inputMonitor & inputMonitor Base



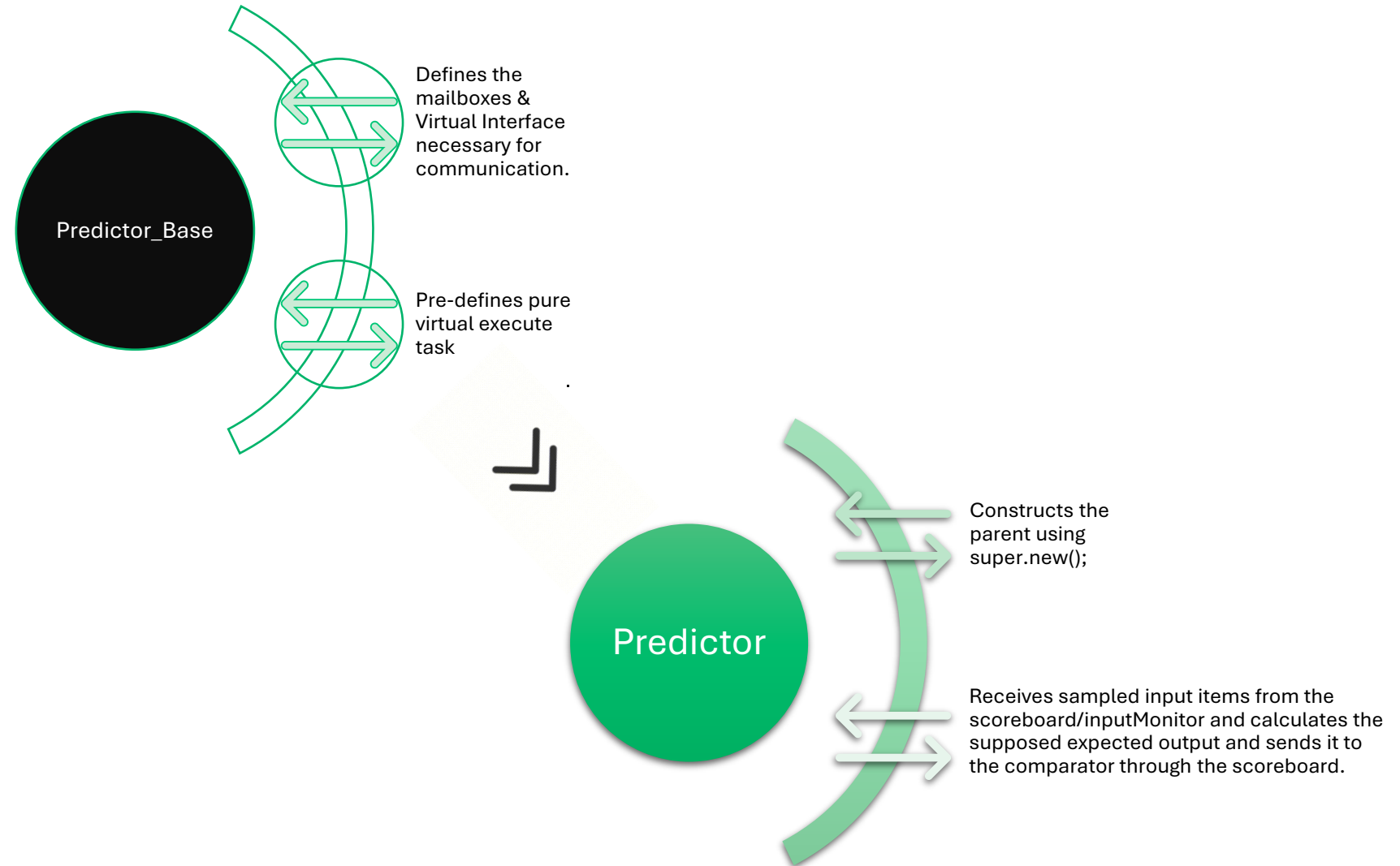
Coverage & Coverage Base



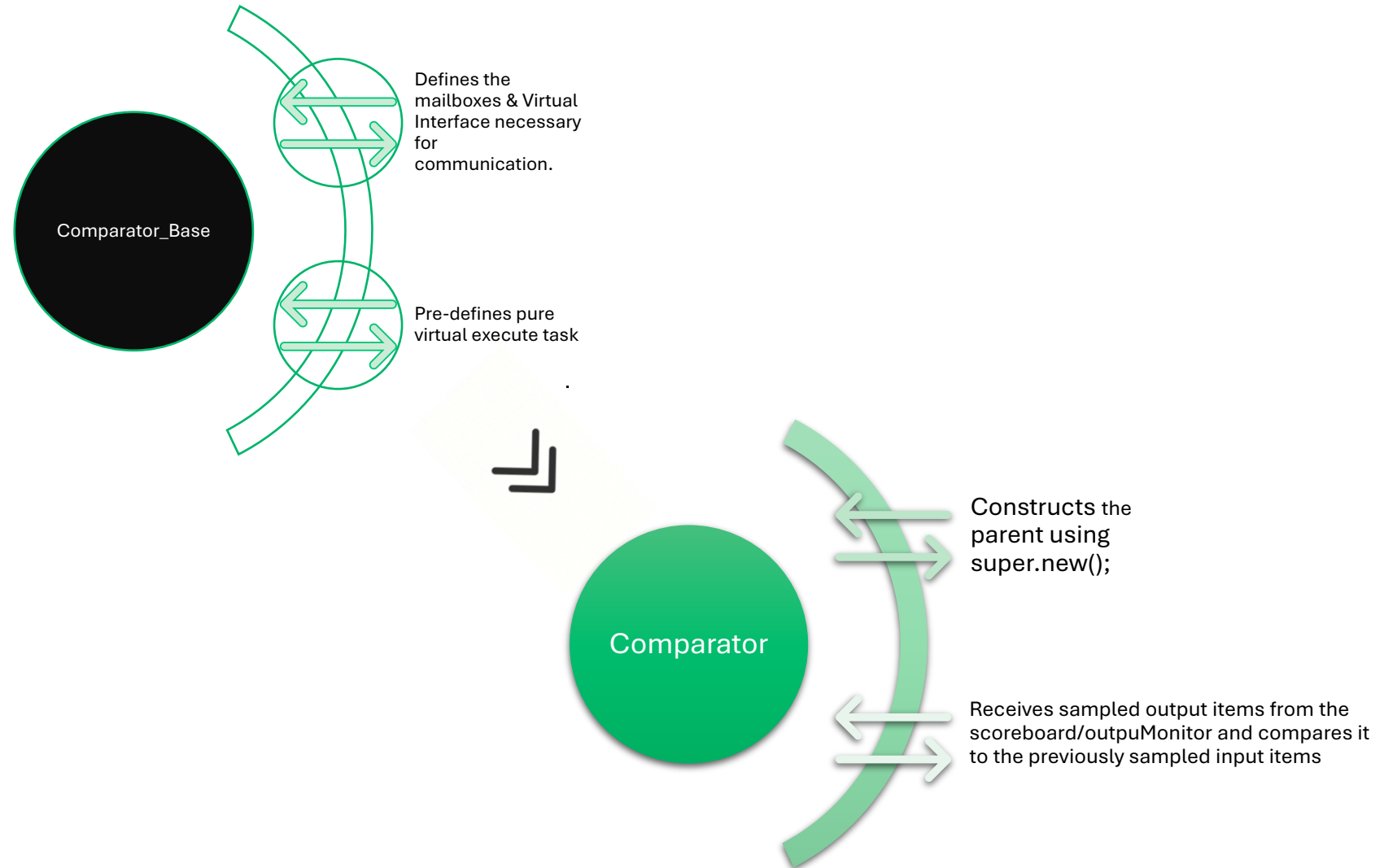
Scoreboard & Scoreboard Base



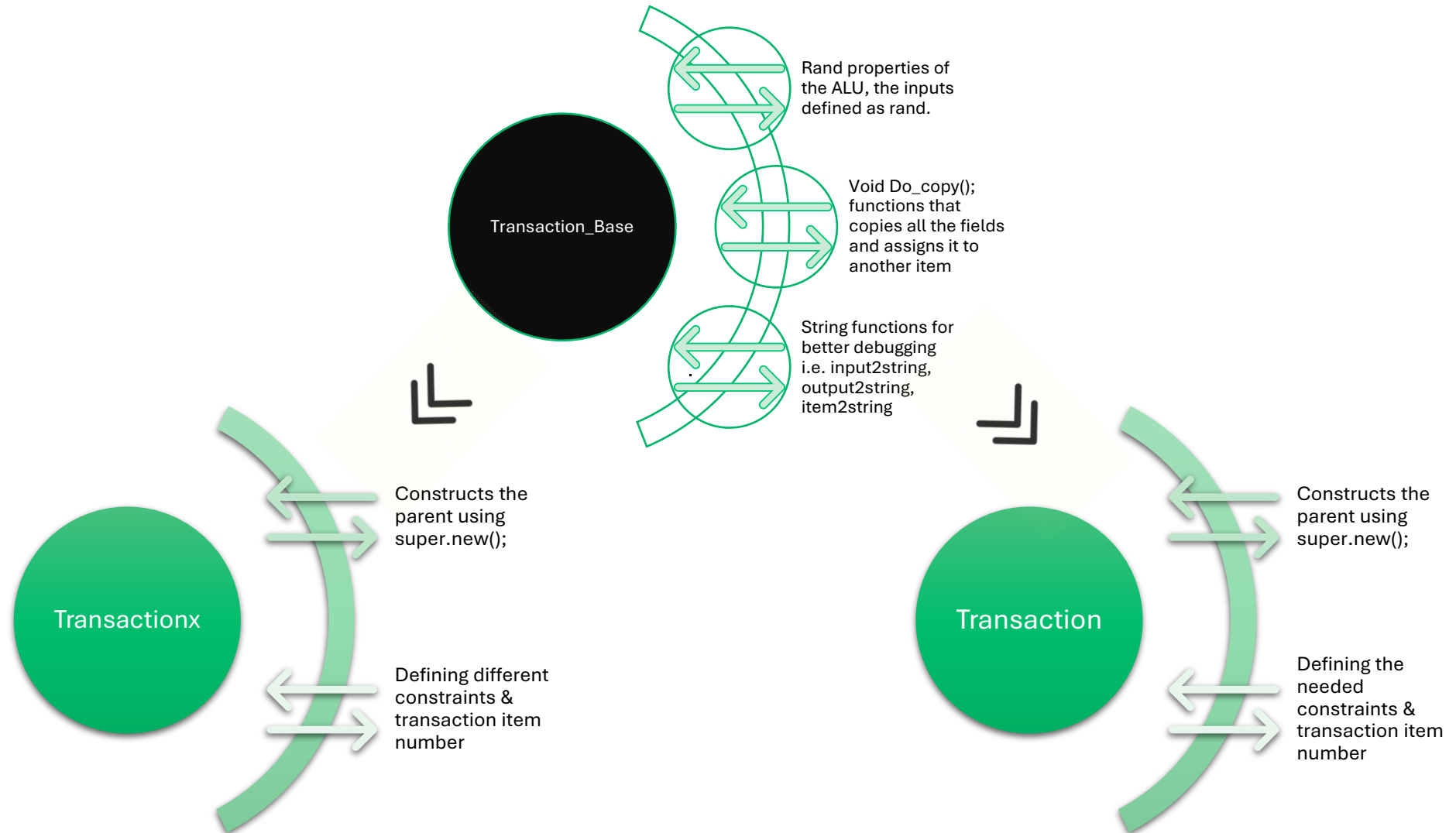
Predictor & Predictor Base



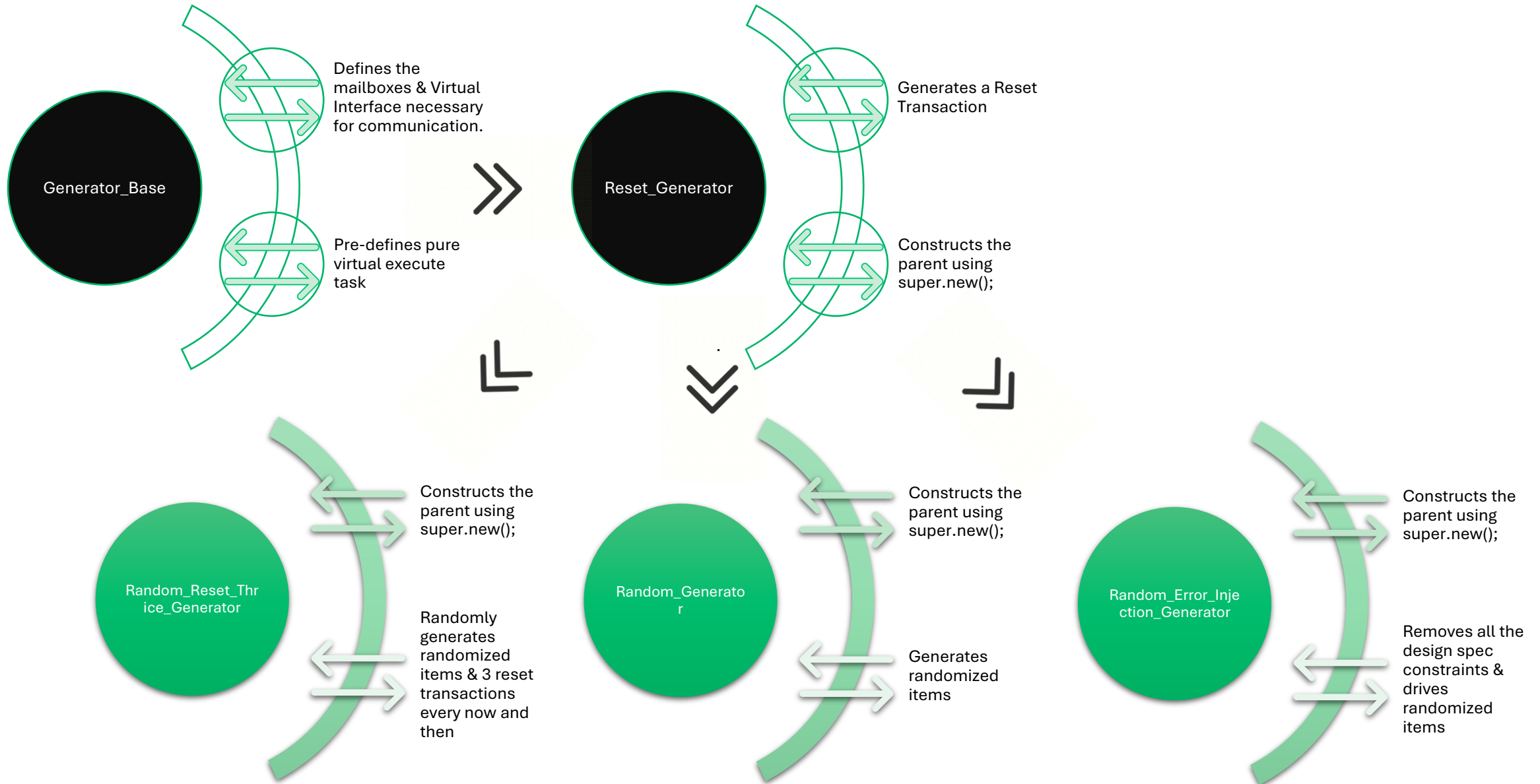
Comparator & Comparator Base



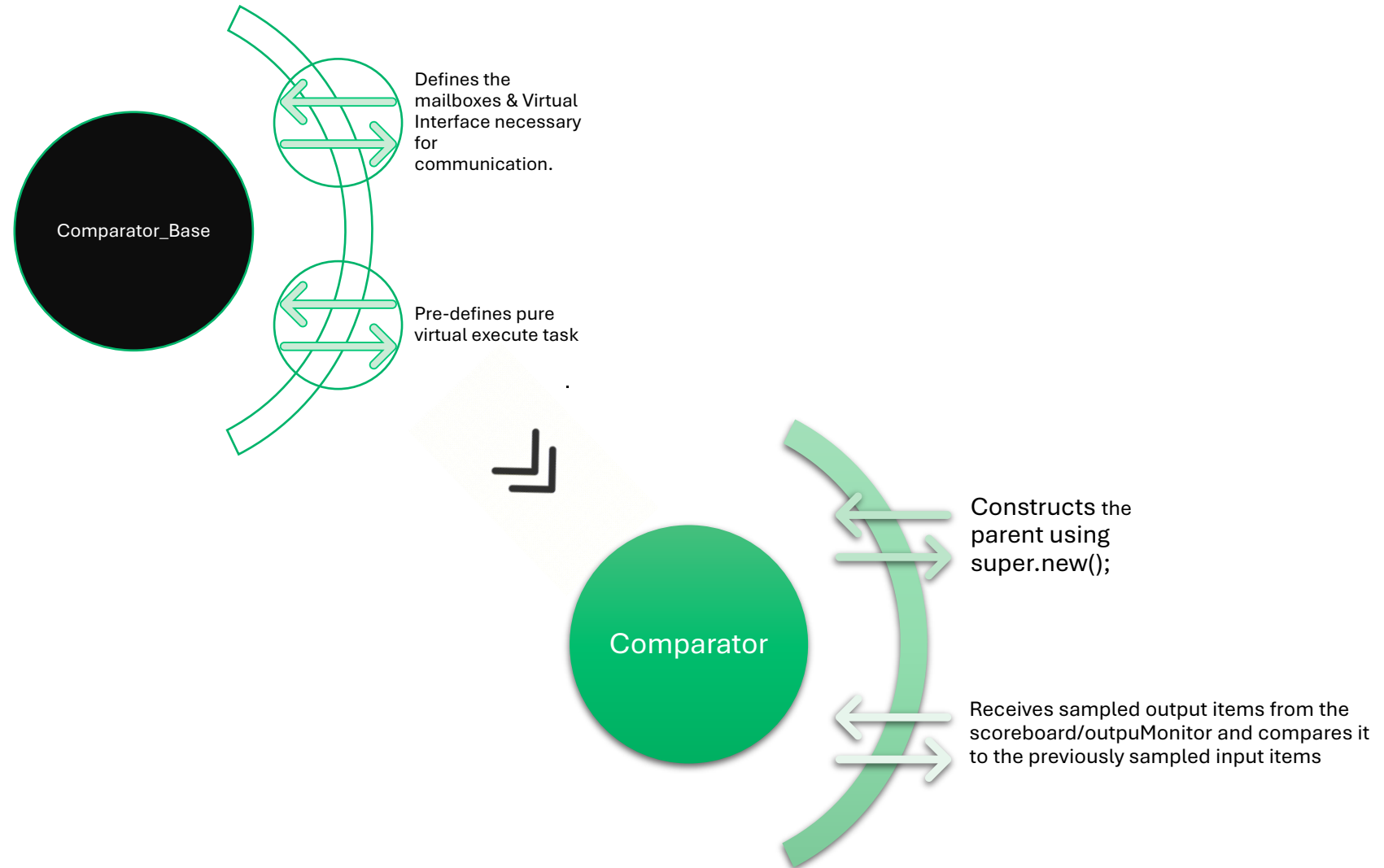
Transaction & Transaction Base



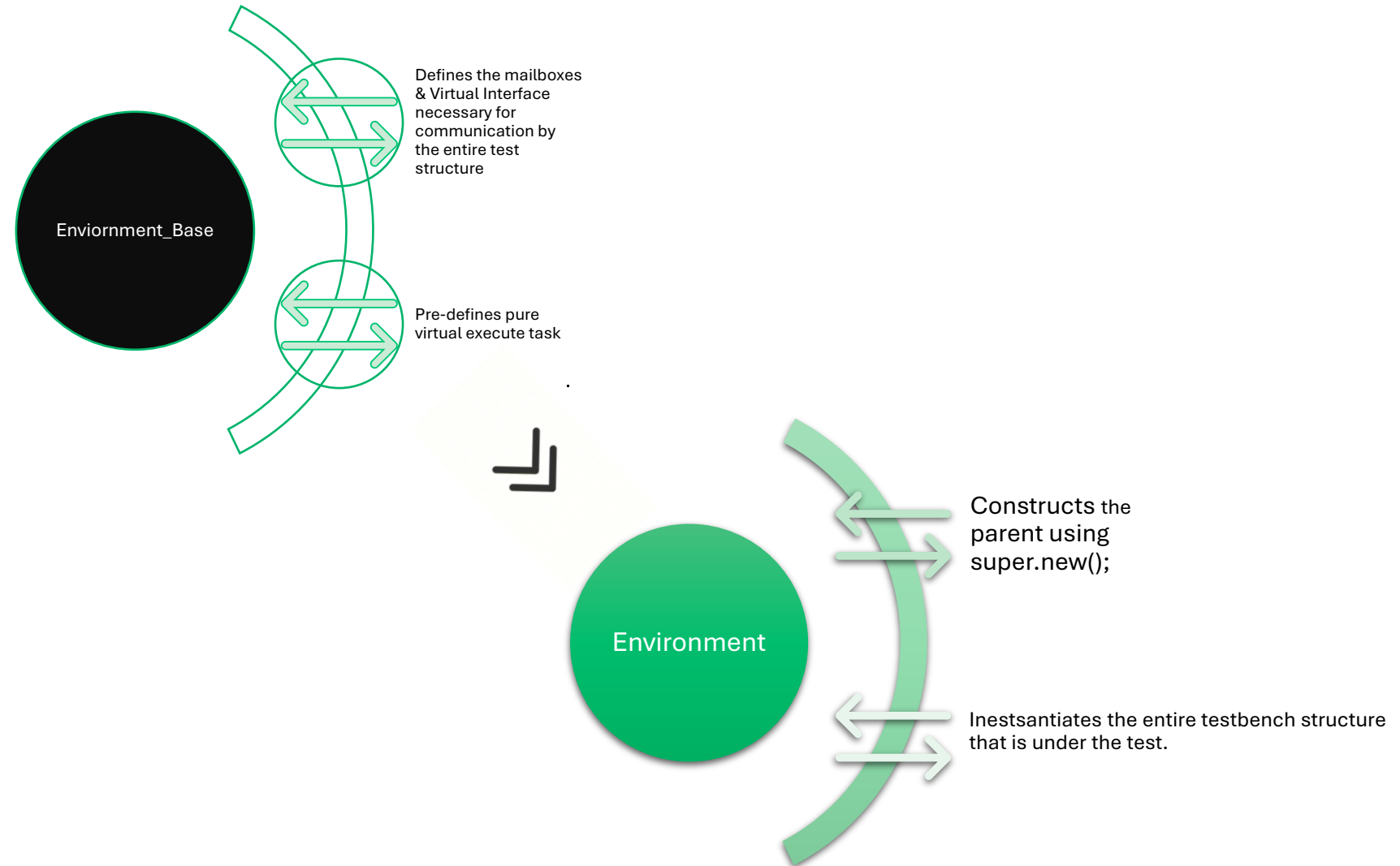
Generator & Generator Base



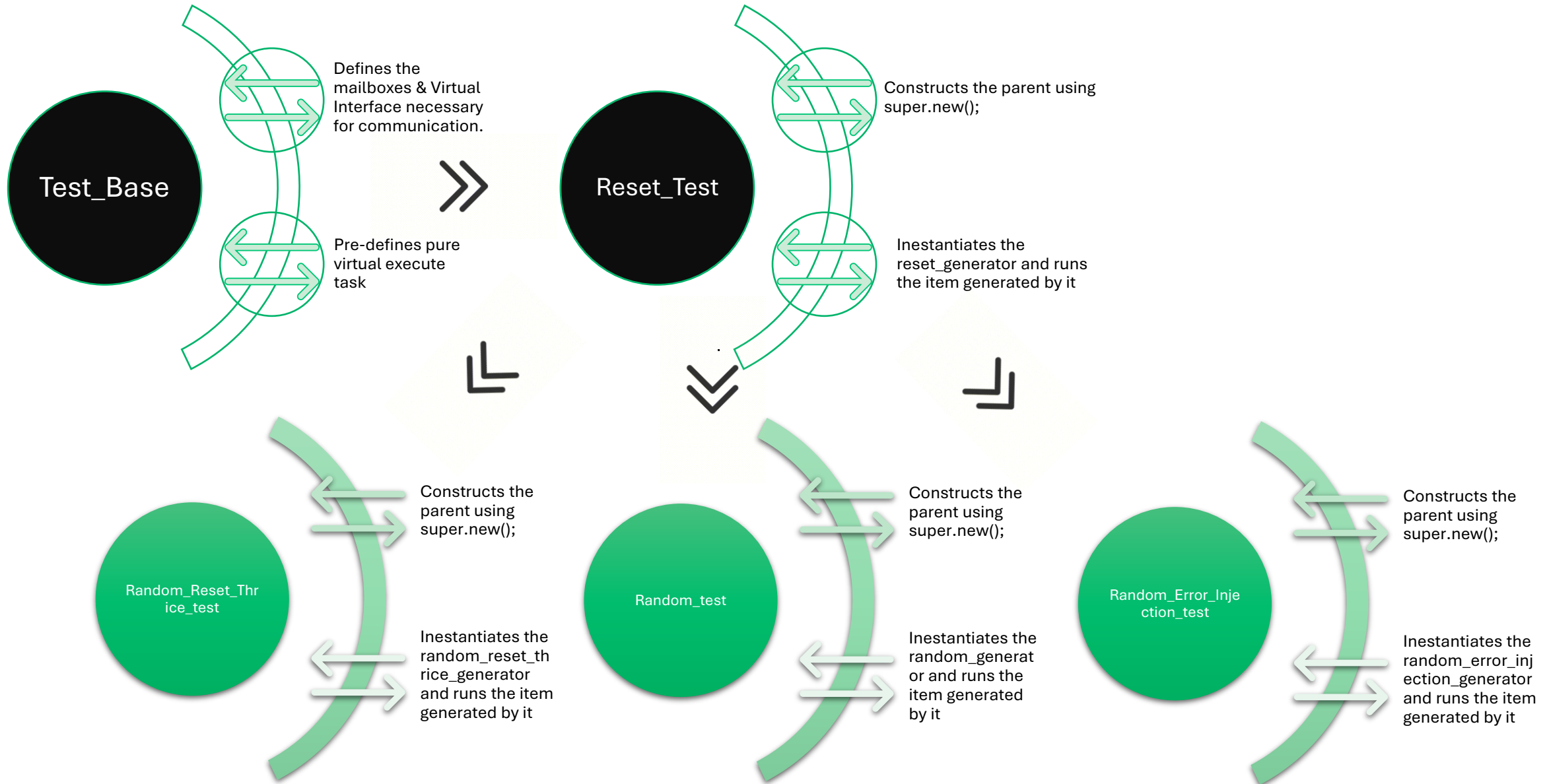
Comparator & Comparator Base



Environment & Environment Base



Test & Test Base



COVERAGE?

What? For What?

100%



Coverage Examples

Fixed/Cross – based coverage method

```
covergroup inputs_cg();
  A_df:coverpoint input_cov_copied.A iff (input_cov_copied.ALU_en & input_cov_copied.rst_n){
    option.weight = ((input_cov_copied.A == -16)?0:1);
    bins A[] = {[ -16:15]};
    `ifdef RANDOM_ERROR_INJECTION_TEST
      ignore_bins A_ignored[] = {-16};
    `else
      illegal_bins A_illegal[] = {-16};
    `endif
  }

  B_df:coverpoint input_cov_copied.B iff (input_cov_copied.ALU_en & input_cov_copied.rst_n){
    option.weight = ((input_cov_copied.B == -16)?0:1);
    bins B[] = {[ -16:15]};
    `ifdef RANDOM_ERROR_INJECTION_TEST
      ignore_bins B_ignored[] = {-16};
    `else
      illegal_bins B_illegal[] = {-16};
    `endif
  }

  A_B_Aen_Ben_Aop_Bop: cross A_df, B_df, A_en_df, B_en_df, A_op_df, B_op01_df, B_op11_df{
    bins A_op_cases = binsof(A_df) && binsof(B_df) && binsof(A_en_df) intersect {1} &&
      binsof(B_en_df) intersect {0} && binsof(A_op_df);
    bins B_op01_cases = binsof(A_df) && binsof(B_df) && binsof(A_en_df) intersect {0} &&
      binsof(B_en_df) intersect {1} && binsof(B_op01_df);
    bins B_op11_cases = binsof(A_df) && binsof(B_df) && binsof(A_en_df) intersect {1} &&
      binsof(B_en_df) intersect {1} && binsof(B_op11_df);
  }
}
```

```
inputs_cg = new();
```

```
outputs_cg = new();
```

```
inputs_cg.sample();
outputs_cg.sample();
```

Instance – based coverage method

```
covergroup A_cg_df(input int signed i, input transaction_base cov);
  option.weight = ((i == -16)? 0:1);
  option.name = $sformatf("df = %0d",i);
  option.per_instance = 1;
  df: coverpoint cov.A iff (cov.rst_n && cov.ALU_en) {
    bins A[] = {i};
    `ifdef RANDOM_ERROR_INJECTION_TEST
      ignore_bins A_ignored[] = {-16};
    `else
      illegal_bins A_illegal[] = {-16};
    `endif
  }
endgroup : A_cg_df
```

```
covergroup A_op_cg_dt(input int i, input int k, input transaction_base cov);
  option.weight = ((k == 7 || i == 7)? 0:1);
  option.name = $sformatf("dt %0d => %0d", i, k);
  option.per_instance = 1;
  dt: coverpoint cov.a_op iff (cov.rst_n && (cov.a_en & ~cov.b_en) && cov.ALU_en) {
    bins A_op[] = {i => k};
    `ifdef RANDOM_ERROR_INJECTION_TEST
      ignore_bins A_op_ignored1[] = {i => 7};
      ignore_bins A_op_ignored2[] = {7 => k};
    `else
      illegal_bins A_op_illegal1[] = {i => 7};
      illegal_bins A_op_illegal2[] = {7 => k};
    `endif
  }
endgroup : A_op_cg_dt
```

```
foreach(A_op_cg_df_vals[i]) A_op_cg_df_vals[i] = new(i, input_cov_copied);
```

```
foreach(A_cg_df_vals[i]) A_cg_df_vals[i].sample();
foreach(B_cg_df_vals[i]) B_cg_df_vals[i].sample();

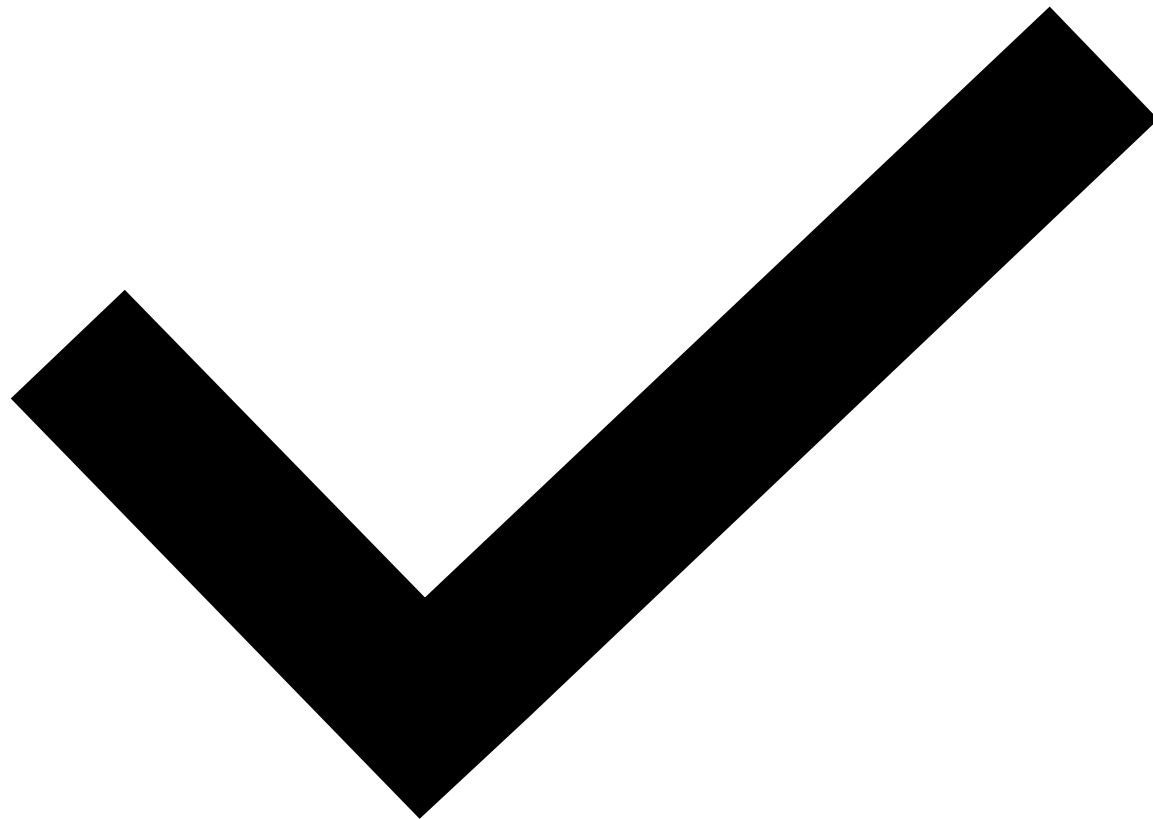
foreach(A_op_cg_dt_vals[i,j]) A_op_cg_dt_vals[i][j].sample();
foreach(B_op01_cg_dt_vals[i,j]) B_op01_cg_dt_vals[i][j].sample();
foreach(B_op11_cg_dt_vals[i,j]) B_op11_cg_dt_vals[i][j].sample();
```

BODY

Construction

Sampling

Coverage Closure



Dashboard.txt for valid Test

```
Dashboard
Date: Thu Feb 20 20:42:28 2025
User: svgpdv25abohamad
Version: V-2023.12-1
Command line: urg -dir RANDOM_TEST_simulation_executable.vdb RANDOM_RESET_THRICE_TEST_simulation_executable.vdb -format text -elfile RANDOM_TEST.el -report coverage_closure/valid_tests_cov_merged.txt

Number of tests: 2

-----
Total Coverage Summary
SCORE LINE COND TOGGLE BRANCH ASSERT GROUP
98.19 100.00 -- 100.00 100.00 92.00 98.96

-----
Hierarchical coverage data for top-level instances
SCORE LINE COND TOGGLE BRANCH ASSERT NAME
100.00 -- -- -- -- 100.00 alu_pkg(x)
97.83 100.00 -- 100.00 100.00 91.30 alu_top_tb(X)

-----
Total Module Definition Coverage Summary
SCORE LINE COND TOGGLE BRANCH ASSERT
98.00 100.00 -- 100.00 100.00 92.00

-----
Total Groups Coverage Summary
SCORE INST SCORE WEIGHT
98.96 98.96 1
```

```
[svgpdv25abohamad@academysvr02 valid_tests_cov_merged.txt]$ ls
asserts.txt groups.txt hierarchy.txt modlist.txt tests.txt
dashboard.txt grpinfo.txt modinfo.txt session.xml
```

Dashboard.txt for Error Injection Test

```
Dashboard
Date: Thu Feb 20 20:42:28 2025
User: svgpdv25abohamad
Version: V-2023.12-1
Command line: urg -dir RANDOM_ERROR_INJECTION_TEST_simulation_executable.vdb -format text -elfile RANDOM_ERROR_INJECTION_TEST.el -report coverage_closure/error_injection_cov_merged.txt

Number of tests: 1

-----
Total Coverage Summary
SCORE LINE COND TOGGLE BRANCH ASSERT GROUP
98.50 100.00 -- 99.38 100.00 93.10 100.00

-----
Hierarchical coverage data for top-level instances
SCORE LINE COND TOGGLE BRANCH ASSERT NAME
50.00 -- -- -- -- 50.00 alu_pkg
99.85 100.00 -- 99.38 100.00 100.00 alu_top_tb(X)

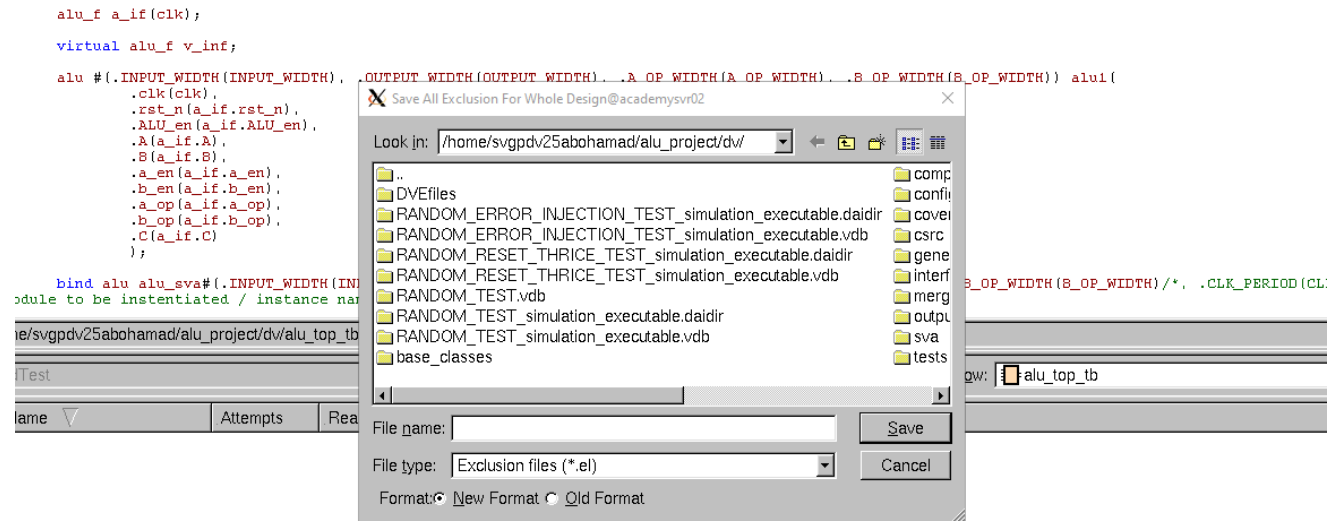
-----
Total Module Definition Coverage Summary
SCORE LINE COND TOGGLE BRANCH ASSERT
93.77 100.00 -- 99.38 82.61 93.10

-----
Total Groups Coverage Summary
SCORE INST SCORE WEIGHT
100.00 100.00 1
```

```
[svgpdv25abohamad@academysvr02 error_injection_cov_merged.txt]$ ls
asserts.txt groups.txt hierarchy.txt modlist.txt tests.txt
dashboard.txt grpinfo.txt modinfo.txt session.xml
```

Exclusions and .el files

Exclusions can be done in the merge command line using `-exclude` but the naming of the branches, points, ..etc is often hard to make out and understand to just do it from the line, therefore using `dve` to make the `.el` file is much easier and thereafter including them in the report command.



Scripting & Makefiles

```
#!/bin/bash

# Create the 'output' directory if it does not exist
if [ ! -d "output" ]; then
    mkdir output
    echo "Directory 'output' created."
else
    echo "Directory 'output' already exists."
fi

if [ ! -d "coverage_closure" ]; then
    mkdir coverage_closure
    echo "Directory 'coverage_closure' created."
else
    echo "Directory 'coverage_closure' already exists."
fi

# Run the make clean command
make -f alu_make.make clean #cleans the entire vdb, daidir, coverage_closure directories

make -f alu_make.make TEST_NAME=RANDOM_TEST full-sim
make -f alu_make.make TEST_NAME=RANDOM_ERROR_INJECTION_TEST full-sim
make -f alu_make.make TEST_NAME=RANDOM_RESET_THRICE_TEST full-sim

make -f alu_make.make TEST_NAME1=RANDOM_TEST TEST_NAME2=RANDOM_RESET_THRICE_TEST TEST_NAME3=RANDOM_ERROR_INJECTION_TEST generate-reports
```

A simple bash script to create
./output and ./coverage_closure
directories, as well as running the
make file

Scripting & Makefiles

```
# Define variables
VCS = vcs
SIM_EXEC = $(TEST_NAME)_simulation_executable # Name of your simulation executable based on TEST_NAME
DESIGN_FILE = dut.f
TESTBENCH_FILE = tb.f
VPD_FILE = $(TEST_NAME).vpd # VPD file name based on TEST_NAME
OUTPUT_DIR = ./output # Output directory based on TEST_NAME
VERDI = verdi
DVE = dve
FILE_LIST_FLAGS = -f
COVERAGE_FLAGS = -cm branch+cond+tgl+line+assert
COMPILE_LOG = -l $(TEST_NAME)_compile.log
SIM_LOG = -l $(TEST_NAME)_sim.log
VCS_FLAGS = -full64 -sverilog $(COVERAGE_FLAGS) $(COMPILE_LOG) -debug_access+r +R
VPD_FLAGS = -vpd $(OUTPUT_DIR)/$(VPD_FILE)
#COMP_DATA_DIR = $(TEST_NAME)_simulation_executable.daidir
COVERAGE_DB = $(SIM_EXEC).vdb
COVERAGE_DIR = coverage_closure
```

```
# Define the default test name (can be overridden via command line)
TEST_NAME ?= RANDOM_TEST # Default test name if not specified
TEST_NAME1 ?= RANDOM_TEST
TEST_NAME2 ?= RANDOM_RESET_THRICE_TEST
TEST_NAME3 ?= RANDOM_ERROR_INJECTION_TEST
```

```
# Compile the design and testbench with the specified test name
```

```
compile: $(DESIGN_FILE) $(TESTBENCH_FILE)
```

Compile the testbench with the correct defines needed using the TEST_NAME variable

```
@echo "Compiling design and testbench for test: $(TEST_NAME)..."
```

```
$(VCS) $(VCS_FLAGS) $(FILE_LIST_FLAGS) $(DESIGN_FILE) $(FILE_LIST_FLAGS) $(TESTBENCH_FILE) +define+$(TEST_NAME) -o $(SIM_EXEC)
```

```
@echo "Compilation complete."
```

Start the simulation using the output file from the compilation according to TEST_NAME variable

```
# Run the simulation and dump the VPD file with the test-specific name
```

```
run: compile
```

```
@echo "Running simulation for test: $(TEST_NAME)..."
```

```
./$(SIM_EXEC) $(COVERAGE_FLAGS) +$(TEST_NAME) +ntb_random automatic $(SIM_LOG) $(VPD_FLAGS)
```

```
@echo "Simulation complete. VPD file dumped to $(OUTPUT_DIR)/$(VPD_FILE)."
```

Run either DVE or VERDI to check the waveform of the VPD file for each TEST

```
# Check if user wants to use Verdi or DVE
```

```
check-viewer:
```

```
@echo "Which waveform viewer would you like to use? (Enter 'verdi' or 'dve'): " ; \
```

```
read viewer; \
```

```
case "$viewer" in \
```

```
verdi) PROG="$(VERDI)" ;; \
```

```
dve) PROG="$(DVE)" ;; \
```

```
*) echo "Invalid option. Please enter either 'verdi' or 'dve'."; exit 1 ;; \
```

```
esac; \
```

```
echo "Which test waveform (VPD file) would you like to use? (e.g., RANDOM_TEST.vpd): " ; \
```

```
read chosen_vpd; \
```

```
# Optionally assign the chosen VPD file name to TEST_NAME for informational purposes
```

```
TEST_NAME="${chosen_vpd}"; \
```

```
echo "Using test waveform: $TEST_NAME"; \
```

```
$$PROG -vpd $(OUTPUT_DIR)/$(VPD_FILE)
```

```
# Generate coverage reports
```

```
generate-reports:
```

```
@echo "Generating coverage reports....."
```

```
urg -dir $(TEST_NAME1)_simulation_executable.vdb $(TEST_NAME2)_simulation_executable.vdb -format text -elfile RANDOM_TEST.el -report $(COVERAGE_DIR)/valid_tests_cov_merged.txt
```

```
urg -dir $(TEST_NAME3)_simulation_executable.vdb -format text -elfile RANDOM_ERROR_INJECTION_TEST.el -report $(COVERAGE_DIR)/error_injection_cov_merged.txt
```

Generating the coverage reports of the valid tests _merging them and of the error_injection test

```
clean:
```

```
@echo "Cleaning up..."
```

```
@rm -rf "$(OUTPUT_DIR)"/* "$(COVERAGE_DIR)"/* \
```

```
"$(TEST_NAME1)_simulation_executable" \
```

```
"$(TEST_NAME2)_simulation_executable" \
```

```
"$(TEST_NAME3)_simulation_executable" \
```

```
"$(TEST_NAME1)_simulation_executable.daidir" \
```

```
"$(TEST_NAME2)_simulation_executable.daidir" \
```

```
"$(TEST_NAME3)_simulation_executable.daidir" \
```

```
"$(TEST_NAME1)_simulation_executable.vdb" \
```

```
"$(TEST_NAME2)_simulation_executable.vdb" \
```

```
"$(TEST_NAME3)_simulation_executable.vdb"
```

```
@echo "Clean up complete."
```

Cleaning the previous run's output files as mentioned in the bash_script slide

```
# Full build and simulation flow
```

```
full-sim: compile run
```

Makefile to compile, run, check the simulation waveform using the .vpd files & clean each time the clean task is called or when the run_all.sh is executed

Saving compile and simulation.log files so they can be searched for any errors using a script if I have the time to make one